

Experiment 1: Markov Decision Process (MDP)

Python Program with Output

```
"""
Experiment 1: Markov Decision Process (MDP) for a Simplified Chess Game
Value Iteration is used to compute the optimal policy for an agent piece
that must reach a goal cell on a small board while avoiding a randomly
moving opponent piece.
"""

import random
from itertools import product

# -----
# 1. Environment definition
# -----
BOARD_SIZE = 4
GOAL = (3, 3)
STEP_COST = -1
WIN_REWARD = 10
LOSE_REWARD = -10
GAMMA = 0.9
THETA = 1e-4

ACTIONS = {
    "UP": (-1, 0),
    "DOWN": (1, 0),
    "LEFT": (0, -1),
    "RIGHT": (0, 1),
}

def in_bounds(pos):
    r, c = pos
    return 0 <= r < BOARD_SIZE and 0 <= c < BOARD_SIZE

def opponent_moves(pos):
    """All legal single-step moves for the opponent piece (including staying)."""
    moves = [pos]
    for dr, dc in ACTIONS.values():
        new_pos = (pos[0] + dr, pos[1] + dc)
        if in_bounds(new_pos):
            moves.append(new_pos)
    return moves

# All states: (agent_pos, opponent_pos) with agent != opponent
all_positions = list(product(range(BOARD_SIZE), range(BOARD_SIZE)))
STATES = [(a, o) for a in all_positions for o in all_positions if a != o]

V = {s: 0.0 for s in STATES}

def is_terminal(state):
    agent, opponent = state
    return agent == GOAL or agent == opponent

def step(state, action):
    """Returns list of (probability, next_state, reward) for a given action."""
    agent, opponent = state
    dr, dc = ACTIONS[action]
    new_agent = (agent[0] + dr, agent[1] + dc)
    if not in_bounds(new_agent):
        new_agent = agent # illegal move keeps agent in place

    if new_agent == GOAL:
        return [(1.0, (new_agent, opponent), WIN_REWARD)]
    if new_agent == opponent:
        return [(1.0, (new_agent, opponent), LOSE_REWARD)]

    # Opponent then takes one random legal step
    possible_opp = opponent_moves(opponent)
    prob = 1.0 / len(possible_opp)
    outcomes = []
    for new_opp in possible_opp:
        if new_agent == new_opp:
            outcomes.append((prob, (new_agent, new_opp), LOSE_REWARD))
        else:
```

```

        outcomes.append((prob, (new_agent, new_opp), STEP_COST))
    return outcomes

# -----
# 2. Value Iteration
# -----
def value_iteration():
    sweep = 0
    while True:
        delta = 0.0
        for s in STATES:
            if is_terminal(s):
                continue
            best_value = float("-inf")
            for a in ACTIONS:
                q_sa = sum(p * (r + GAMMA * V.get(s2, 0.0)) for p, s2, r in step(s, a))
                best_value = max(best_value, q_sa)
            delta = max(delta, abs(best_value - V[s]))
            V[s] = best_value
        sweep += 1
        if delta < THETA:
            break
    return sweep

def extract_policy():
    policy = {}
    for s in STATES:
        if is_terminal(s):
            continue
        best_action, best_value = None, float("-inf")
        for a in ACTIONS:
            q_sa = sum(p * (r + GAMMA * V.get(s2, 0.0)) for p, s2, r in step(s, a))
            if q_sa > best_value:
                best_value, best_action = q_sa, a
        policy[s] = best_action
    return policy

# -----
# 3. Simulation
# -----
def simulate(policy, start_state, max_moves=20):
    state = start_state
    print(f"Start: agent={state[0]}, opponent={state[1]}")
    moves = 0
    while not is_terminal(state) and moves < max_moves:
        action = policy[state]
        outcomes = step(state, action)
        probs = [o[0] for o in outcomes]
        idx = random.choices(range(len(outcomes)), weights=probs, k=1)[0]
        _, state, reward = outcomes[idx]
        moves += 1
        print(f"Move {moves}: action={action:<5} -> agent={state[0]}, opponent={state[1]}, reward={reward}")
    if state[0] == GOAL:
        print("Result: Agent reached the goal. WIN!")
    elif state[0] == state[1]:
        print("Result: Agent was captured. LOSE!")
    else:
        print("Result: Move limit reached. DRAW.")

if __name__ == "__main__":
    random.seed(7)
    sweeps = value_iteration()
    print(f"Value Iteration converged in {sweeps} sweeps.\n")
    optimal_policy = extract_policy()
    start = ((0, 0), (2, 1))
    simulate(optimal_policy, start)

```

Program Output

Value Iteration converged in 15 sweeps.

Start: agent=(0, 0), opponent=(2, 1)

Move 1: action=RIGHT -> agent=(0, 1), opponent=(1, 1), reward=-1

Move 2: action=RIGHT -> agent=(0, 2), opponent=(1, 1), reward=-1

Move 3: action=RIGHT -> agent=(0, 3), opponent=(1, 0), reward=-1

Move 4: action=DOWN -> agent=(1, 3), opponent=(1, 0), reward=-1

Move 5: action=DOWN -> agent=(2, 3), opponent=(2, 0), reward=-1

Move 6: action=DOWN -> agent=(3, 3), opponent=(2, 0), reward=10

Result: Agent reached the goal. WIN!